

Logique et raisonnement

CM7 : Logique et modèles de premier ordre

Catalin Dima, UPEC

2021

Logiques de premier ordre

► Pas d'algorithme qui, étant donnés :

- un ensemble de fonctions $\mathcal{F}$ (attention ! notations modifiées !),
- un ensemble de relations $\mathcal{R}$,
- une formule ϕ ,

décide si la formule ϕ est satisfaisable.

► Pas d'algorithme qui, étant donnés :

- un ensemble de prémisses Γ ,
- une formule ϕ ,

décide si ϕ peut être déduite de Γ (càd, il existe une preuve).

- Pas d'algorithme même si on se restreint à $\mathcal{F} = \{0, s, +, \cdot\}$,
 $\mathcal{R} = \{'' = ''\}$ et aux axiomes de l'arithmétique de Peano.
- Pourtant, si on se restreint à l'arithmétique de Presburger, un tel algorithme existe.
- D'autres exemples de restrictions qui rendent les problèmes de satisfaisabilité et de prouvabilité décidables ?

Calcul relationnel

Définition

Le **calcul relationnel** est la logique de premier ordre dont le langage est composé d'un ensemble de fonctions $\mathcal{F}$ qui ne contient que des **constantes**.

- ▶ Formellement, pour tout $f \in \mathcal{F}$, $ar(f) = 0$.
- ▶ Pas de contrainte sur les relations.
- ▶ À quoi ça ressemble ?

Calcul relationnel

Définition

Le **calcul relationnel** est la logique de premier ordre dont le langage est composé d'un ensemble de fonctions $\mathcal{F}$ qui ne contient que des **constantes**.

- ▶ Formellement, pour tout $f \in \mathcal{F}$, $ar(f) = 0$.
- ▶ Pas de contrainte sur les relations.
- ▶ À quoi ça ressemble ?
 1. $\forall x(A(x, a) \rightarrow T(a, x))$
 2. $\forall x \exists y(A(y, x) \wedge T(x, y))$
 3. $\neg \exists x \forall y(A(y, a) \rightarrow T(x, y))$
 4. $\forall x \forall y(T(x, y) \rightarrow A(x, y))$
- ▶ "Alice a le téléphone de tous ses amis".
- ▶ Etc.

Exemple de **requêtes de bases de données** !

Calcul relationnel et algèbre relationnelle

Est-ce que toute formule du calcul relationnel correspond à une requête de l'algèbre relationnelle ?

- ▶ Se rappeler que les requêtes SQL sont souvent associés à des domaines (tables).
- ▶ Ces tables sont **finies** : restriction sémantique du calcul relationnel.

Interprétations du calcul relationnel

On ne considère que les interprétations qui associent un ensemble **fini** à chaque symbole relationnel $R \in \mathcal{R}$, $\text{card}(i(R)) < \infty$.

Calcul relationnel et algèbre relationnelle

Est-ce que toute formule du calcul relationnel correspond à une requête de l'algèbre relationnelle ?

- ▶ Se rappeler que les requêtes SQL sont souvent associés à des domaines (tables).
- ▶ Ces tables sont **finies** : restriction sémantique du calcul relationnel.

Interprétations du calcul relationnel

On ne considère que les interprétations qui associent un ensemble **fini** à chaque symbole relationnel $R \in \mathcal{R}$, $\text{card}(i(R)) < \infty$.

- ▶ Même avec cette restriction, certaines formules du calcul relationnel sont "trop vagues" :
- ▶ Quels sont les films de Hitchcock dans lesquels Hitchcock n'a pas joué ?

$$\phi : \neg \text{Film}(x, \text{"Hitch"}, \text{"Hitch"})$$

- ▶ Remarquer que x n'est pas quantifié.
- ▶ ϕ peut s'interpréter (dans un modèle) comme l'ensemble des valeurs pour x qui satisfont ϕ : **l'ensemble retourné par ϕ** .
- ▶ Par rapport à quelle table (= domaine) on interprète x ? Tous les acteurs du monde ?...

Calcul relationnel versus algèbre relationnelle ?

- ▶ Intuitivement, les formules qui sont modélisables en algèbre relationnelle (et donc en requêtes SQL) ne doivent pas dépendre de la connaissance d'un "univers des valeurs".
- ▶ **Indépendance de domaine** :
 - ▶ Une formule du calcul relationnel est **sûre** si elle "retourne" un ensemble fini lorsque l'interprétation de chaque symbole relationnel est une relation de cardinalité finie.
 - ▶ Noter que le domaine d'interprétation D peut être infini !
- ▶ Contre-exemple : $\phi : \neg Film(x, "Hitch", "Hitch")$:
 - ▶ En prenant $D =$ les films faits jusqu'à aujourd'hui **plus** tous les entiers (oui !), ϕ sera vraie pour tous les entiers !
- ▶ Bon exemple : $\psi : \exists y Film(x, "Hitch", y) \wedge \neg Film(x, "Hitch", "Hitch")$.
 - ▶ Tous les films qui ont été réalisés par Hitchcock dans lesquels il n'a pas joué.
 - ▶ Quelque soit D , vu que $Film$ sera interprétée comme une relation finie sur D , $\{x \mid \psi(x)\}$ sera toujours un ensemble fini !

Calcul relationnel "simple"

Restriction syntaxique : on n'accepte que des formules de trois types :

1. Des **faits** : formules simples $R(v_1, \dots, v_k)$ ($k = ar(R)$ est l'arité de R) où $v_1, \dots, v_k \in D$ (constantes).
 - ▶ Mélange de syntaxe et sémantique.
2. Des **règles** : implications de type $\forall x_1 \dots \forall x_n (R_1 \wedge \dots \wedge R_n \rightarrow T)$. dans lesquelles toutes les R_i et T sont des **formules atomiques** utilisant les variables $x_1, \dots, x_n$ (et pas d'autres).
3. Des **buts** : conjonctions quantifiées de type $\exists x_1 \dots \exists x_n (T_1(x_1, \dots, x_k) \wedge \dots \wedge T_n(x_1, \dots, x_n))$ avec la même contrainte sur toutes les ϕ_i .
 - ▶ Les formules de type 1 sont des **faits**.
 - ▶ Les formules de type 2 sont des **règles définissant** le symbole relationnel T correspondant.
 - ▶ La formule de type 3 est **la formule but**.

Datalog positif non-récuratif

Définition

Un **programme** de type **Datalog positif non-récuratif** est un formule

$\phi_1 \wedge \dots \phi_n \rightarrow \psi$ où :

1. ϕ_i sont de type 1 (faits) ou 2 (règles).
2. ψ est une formule de type 3 (but).
3. Pas de définition par une règle des relations R utilisées dans les faits.
4. Pas de négation.
5. Pas de **cycles** dans les définitions des symboles T dans les règles.
 - ▶ Pas de règle définissant T_1 qui utilise T_2 , alors que la règle définissant T_2 utilise T_1 .
 - ▶ Ou d'autres cycles de longueur plus grande.

Datalog positif non-récuratif

Exemple de programme Datalog positif non-récuratif :

```
(student(aa001, john, mr.) ∧  
  student(aa002, mary, mme.) ∧  
  student(aa003, ann, mme.) ∧  
  study(aa001, law) ∧  
  study(aa002, maths) ∧  
  study(aa003, info) ∧  
  ∀x study(x, info) → sciencestud(x) ∧  
  ∀x study(x, maths) → sciencestud(x))  
→ ∃x ∃y student(x, y, mme.), sciencestud(x)
```

Exemple de formule qui n'est pas un programme Datalog positif non-récuratif :

```
(parent(alice, bob) ∧  
  parent(bob, charlie) ∧  
  ∀x ∀y ∀z (parent(x, y) ∧ ancetre(y, z) → ancetre(x, z)))  
→ ancetre(alice, charlie)
```

car définition "réursive" de la relation "ancêtre".

Datalog positif non-récuratif et SQL

Théorème

Le **Datalog positif non-récuratif** est équivalent aux requêtes **SQL** simples (variante **sélection/projection/produit cartésien** ou **sélection/projection/jointure/renommage**).

Il existe un algorithme qui permet de décider si un programme Datalog positif non-récuratif est satisfaisable.

Datalog positif non-récuratif et SQL

Théorème

Le **Datalog positif non-récuratif** est équivalent aux requêtes **SQL** simples (variante **sélection/projection/produit cartésien** ou **sélection/projection/jointure/renommage**).

Il existe un algorithme qui permet de décider si un programme Datalog positif non-récuratif est satisfaisable.

Idée d'algorithme :

- ▶ On prend les faits comme "tables de base".
- ▶ On construit les "tables auxiliaires" en y insérant les tuples définis par les règles.
- ▶ On résout la contrainte but par "parcours des tables".

Pertinence de l'algorithme (et donc du théorème !) :

- ▶ Évite la construction des produits cartésiens (gourmande en temps et espace).
- ▶ Problème similaire pour la jointure.
- ▶ Toute l'algorithmique "rapide" du SQL repose sur des améliorations de cet algorithme.

Datalog positif non-récuratif formaté

$$\begin{aligned} & \left(\text{student}(\text{aa001}, \text{john}, \text{mr.}) \wedge \right. \\ & \quad \text{student}(\text{aa002}, \text{mary}, \text{mme.}) \wedge \\ & \quad \text{student}(\text{aa003}, \text{ann}, \text{mme.}) \wedge \\ & \quad \text{study}(\text{aa001}, \text{law}) \wedge \\ & \quad \text{study}(\text{aa002}, \text{maths}) \wedge \\ & \quad \text{study}(\text{aa003}, \text{info}) \wedge \\ & \quad \forall x \text{ study}(x, \text{info}) \rightarrow \text{sciencestud}(x) \wedge \\ & \quad \left. \forall x \text{ study}(x, \text{maths}) \rightarrow \text{sciencestud}(x) \right) \\ & \rightarrow \exists x \exists y \text{ student}(x, y, \text{mme.}), \text{sciencestud}(x) \end{aligned}$$

Formaté :

```
student(aa001, john, m).
student(aa002, mary, f).
student(aa003, ann, f).
study(aa001, law).
study(aa002, maths).
study(aa003, info).
sciencestud(X) :- study(X, info).
sciencestud(X) :- study(X, maths).
query(X,Y) :- student(X, Y, F), sciencestud(X).
query(X,Y) ?
```

Programme AbcDatalog

<https://abcdatalog.seas.harvard.edu/>

- ▶ Suite de faits et des règles, chacune terminée par un **point**.
- ▶ Les variables commencent par des majuscules ou `_` (tiret du bas, underscore)
- ▶ Les constantes et les symboles relationnels commencent par des minuscules.
- ▶ Variable "anonyme" `_`
- ▶ Test d'égalité admis entre variables et/ou constantes.
- ▶ Les règles $\phi_1 \wedge \dots \wedge \phi_k \rightarrow \psi$ formatées comme suit :
 - ▶ ψ :- $\phi_1, \phi_2, \dots, \phi_k$.
 - ▶ Sans quantificateurs,
 - ▶ ":-" représente l'implication dans une règle, en "écriture inversée".
 - ▶ Conjonctions remplacées par virgules.
 - ▶ ψ est la **tête** de la règle.
 - ▶ $\phi_1, \dots, \phi_k$ sont le **corps** de la règle.
- ▶ Une seule formule but (sans quantificateurs).
 - ▶ Mais si on a besoin d'une formule but plus complexe, on peut introduire un symbole relationnel supplémentaire qui devient le but.

- Règles multiples pour la même relation R = disjonctions :

```
sciencestud(X) :- study(X, info) .
```

```
sciencestud(X) :- study(X, maths) .
```

Un étudiant en sciences doit étudier soit les maths soit l'info.

- Variables non-liées dans les prémisses = quantification existentielle :

```
sciencestud(X) :- student(X, Y, Z) , study(X, info)
```

Qqun qui étudie en sciences doit être un étudiant (donc

$\exists Y \exists Z \text{ student}(X, Y, Z)$) qui étudie l'info.

Évaluation des programmes Datalog

Algorithme "naïf" de complétion de la **base des faits**.

- ▶ Initialement la **base des faits** est constituée que des faits déclarés dans le programme.
- ▶ Dans un ordre quelconque, on prend une règle et on l'applique à la base des faits.
- ▶ Appliquer une règle qui a $r(X, Y)$ comme tête revient à instancier $X = v_1$ et $Y = v_2$ avec des constantes existantes dans la base des faits, puis vérifier si le corps de la règle est satisfait.
 - ▶ Lorsque, dans le corps on trouve une autre variable Z , on lui affecte une valeur dans les constantes.
 - ▶ Du coup il faut essayer avec toutes les affectations pour Z !
- ▶ Si, pour une telle affectation $X = v_1, Y = v_2$ on vérifie le corps de la règle, on **rajoute le tuple $r(v_1, v_2)$ à la base des faits**.
- ▶ On s'arrête lorsque plus aucune règle ne produit de nouveau fait.

Pseudo-théorème

Tout programme Datalog positif non-récursif est correctement évalué par l'algorithme naïf et donne la "bonne" réponse.

Négation en Datalog

Rappel :

- ▶ Une formule du calcul relationnel est **sûre** si elle "retourne" un ensemble fini lorsque l'interprétation de chaque symbole relationnel est une relation de cardinalité finie.

Théorème

L'ensemble des programmes Datalog avec négation qui sont **sûrs** est équivalent à l'algèbre relationnelle.

- ▶ Càd, **sélection/projection/produit cartésien** ou **sélection/projection/jointure/renommage** plus **différence d'ensembles**.

Problème :

- ▶ Comment tester qu'un programme Datalog est sûr ?
- ▶ Mauvaise nouvelle : il n'existe pas d'algorithme pour vérifier qu'un programme Datalog est sûr !
- ▶ Solution : imposer des restrictions syntaxiques (encore).

Négation en Datalog : restriction de plage pour les variables

Règle de restriction de plage

Dans une règle Datalog (avec négation), on dit qu'une variable X qui apparaît dans un atome `not r(..., X, ...)` satisfait la **restriction de plage** si elle apparaît, dans la même règle, dans un autre atome sans négation.

- ▶ Règle qui satisfait la restriction de plage :

```
nonapparition(X) :- film(X,hitchcock,Y), not film(X, hitchcock, hitchcock).
```

- ▶ Règle incorrecte :

```
nonapparition(X) :- not film(X, hitchcock, hitchcock).
```

- ▶ On voit bien l'idée de restreindre le domaine de valeurs de X aux seules constantes existantes dans les faits du programme !

Datalog non-récuratif et algèbre relationnelle

Théorème

Le **Datalog non-récuratif** est équivalent à l'algèbre relationnelle.

Il existe un algorithme qui permet de décider si un programme Datalog positif non-récuratif est satisfaisable.

L'algorithme est, en fait, le même que celui pour le Datalog positif non-récuratif !

- ▶ La propriété de restriction de plage assure le fait que, lors de l'essai de satisfaction de toute règle, chaque variable recevra une valeur par instantiation avec des constantes dans la base des faits par l'évaluation de l'atome positif où elle apparaît.

Règles récursives

- ▶ Base de données des "amitiés".
- ▶ Mais "les amis de mes amis sont mes amis" !
- ▶ Comment on peut compléter notre table ?

```
ami(joe,sue) .  
ami(ann,sue) .  
ami(sue,max) .  
ami(max,ann) .  
cercle(X,Y) :- ami(X,Y) .  
cercle(X,Z) :- ami(X,Y), cercle(Y,Z) .
```

- ▶ Ne satisfait pas la contrainte de non-récursion pour la relation `cercle` !
- ▶ Mais l'interprétation est assez simple : calculer la **fermeture transitive** de la relation `ami` !

Datalog positif

- ▶ Règles récursives acceptées.
- ▶ Mêmes des récursions "croisées" :
 - ▶ Définition de r qui dépend de t ,
 - ▶ Et définition de t qui dépend de r .
- ▶ Strictement **plus général** que l'algèbre relationnelle !
- ▶ Pour la négation on verra plus tard.

Théorème

La satisfaisabilité d'un programme Datalog positif est décidable.

- ▶ L'algorithme naïf de complétion de la base des faits est encore une fois la solution !
- ▶ Le modèle construit par cet algorithme est inclus dans tout modèle.

Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(X,Y) :- ami(X,Y) .  
cercle(X,Z) :- ami(X,Y) , cercle(Y,Z) .
```

Première itération :

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .
```

Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(X,Y) :- ami(X,Y) .  
cercle(X,Z) :- ami(X,Y) , cercle(Y,Z) .
```

Deuxième itération :

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(joe,sue) .
```


Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(X,Y) :- ami(X,Y) .  
cercle(X,Z) :- ami(X,Y) , cercle(Y,Z) .
```

3e itération :

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(joe,sue) . cercle(ann,sue) .
```

Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(X,Y) :- ami(X,Y) .  
cercle(X,Z) :- ami(X,Y) , cercle(Y,Z) .
```

4e itération :

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(joe,sue) . cercle(ann,sue) .  
cercle(sue,max) .
```

Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(X,Y) :- ami(X,Y) .  
cercle(X,Z) :- ami(X,Y) , cercle(Y,Z) .
```

5e itération (2e règle!) :

```
ami(joe,sue) . ami(ann,sue) .  
ami(sue,max) . ami(max,ann) .  
cercle(joe,sue) . cercle(ann,sue) .  
cercle(sue,max) . cercle(joe,max) .
```

Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue). ami(ann,sue).  
ami(sue,max). ami(max,ann).  
cercle(X,Y) :- ami(X,Y).  
cercle(X,Z) :- ami(X,Y), cercle(Y,Z).
```

6e itération :

```
ami(joe,sue). ami(ann,sue).  
ami(sue,max). ami(max,ann).  
cercle(joe,sue). cercle(ann,sue).  
cercle(sue,max). cercle(joe,max).  
cercle(max,ann).
```

Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue). ami(ann,sue).  
ami(sue,max). ami(max,ann).  
cercle(X,Y) :- ami(X,Y).  
cercle(X,Z) :- ami(X,Y), cercle(Y,Z).
```

7e itération :

```
ami(joe,sue). ami(ann,sue).  
ami(sue,max). ami(max,ann).  
cercle(joe,sue). cercle(ann,sue).  
cercle(sue,max). cercle(joe,max).  
cercle(max,ann). cercle(ann,max).
```

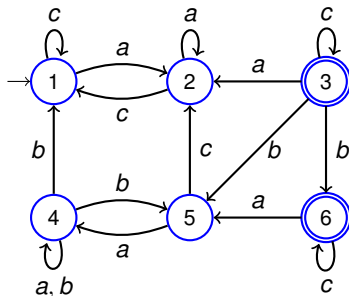
Construction du modèle minimal d'un programme Datalog

```
ami(joe,sue). ami(ann,sue).  
ami(sue,max). ami(max,ann).  
cercle(X,Y) :- ami(X,Y).  
cercle(X,Z) :- ami(X,Y), cercle(Y,Z).
```

12e itération :

```
ami(joe,sue). ami(ann,sue).  
ami(sue,max). ami(max,ann).  
cercle(joe,sue). cercle(ann,sue).  
cercle(sue,max). cercle(joe,max).  
cercle(max,ann). cercle(ann,max).  
cercle(ann,ann). cercle(max,max).  
cercle(sue,sue). cercle(sue,ann).  
cercle(joe,ann). cercle(max,sue).
```

Problème de langage vide d'un automate fini



```
initial(1). final(3). final(6).  
arc(1,c,1). arc(1,a,2). arc(2,a,2). arc(2,c,1).  
arc(3,a,2). arc(3,c,3). arc(3,b,5). arc(3,b,6).  
arc(4,b,1). arc(4,a,4). arc(4,b,4). arc(4,b,5).  
arc(5,a,4). arc(5,c,2). arc(6,a,5). arc(6,c,6).  
accepte :- initial(X), chemin(X,Y), final(Y).  
chemin(X,Y) :- arc(X,A,Y).  
chemin(X,Y) :- chemin(X,Z), arc(Z,Y).
```

Problème de langage vide d'un automate fini

```
initial(1). final(3). final(6).  
arc(1,c,1). arc(1,a,2). arc(2,a,2). arc(2,c,1).  
arc(3,a,2). arc(3,c,3). arc(3,b,5). arc(3,b,6).  
arc(4,b,1). arc(4,a,4). arc(4,b,4). arc(4,b,5).  
arc(5,a,4). arc(5,c,2). arc(6,a,5). arc(6,c,6).  
accepte :- initial(X), chemin(X,Y), final(Y).  
chemin(X,Y) :- arc(X,A,Y).  
chemin(X,Y) :- chemin(X,Z), arc(Z,Y).
```

- ▶ Là on voit le défaut de Datalog : pas de possibilité de **construire** un mot accepté !
- ▶ **Prolog** : Datalog avec des symboles de fonctions !

Datalog récursif avec négation

- ▶ On aimerait rajouter la règle de la restriction de plage à Datalog.
- ▶ Sauf que cela ne marche pas en général !
- ▶ Solution : **stratification**.
 - ▶ Première strate : on peut appliquer la négation qu'aux faits ou aux relations positives.
 - ▶ Deuxième strate : on peut appliquer la négation aux faits ou aux relations dans la première strate.
 - ▶ ...
 - ▶ n -ième strate : on peut appliquer la négation aux relations de strate plus petite que n .
- ▶ Cela permet de construire le modèle le plus petit par le même algorithme.
- ▶ AbcDatalog vérifie si votre programme est stratifié.

Complément d'un graphe

Même graphe (automate), mais maintenant on cherche les paires de noeuds (x, y) tels qu'il **n'existe pas de chemin de x à y** :

```
arc(1,c,1) . arc(1,a,2) . arc(2,a,2) . arc(2,c,1) .  
arc(3,a,2) . arc(3,c,3) . arc(3,b,5) . arc(3,b,6) .  
arc(4,b,1) . arc(4,a,4) . arc(4,b,4) . arc(4,b,5) .  
arc(5,a,4) . arc(5,c,2) . arc(6,a,5) . arc(6,c,6) .  
chemin(X,Y) :- arc(X,A,Y) .  
chemin(X,Y) :- chemin(X,Z) , arc(Z,Y) .  
noeud(X) :- arc(X,A,Y) .  
noeud(Y) :- arc(X,A,Y) .  
nonchemin(X,Y) :- noeud(X) , noeud(Y) , not chemin(X,Y) .
```

Programme stratifiable :

- ▶ Première strate : les relations `arc`, `noeud` et `chemin`.
- ▶ Deuxième strate : la relation `nonchemin`.

Quelques conclusions

- ▶ La logique fournit un (même le !) langage formel permettant de construire les maths et l'informatique :
 - ▶ **Universalité.**
 - ▶ Formalisation des raisonnements.
 - ▶ Formalisation de la notion de modélisation.
- ▶ Élegance mais difficultés algorithmiques.
- ▶ Fragments logiques :
 - ▶ Théories logiques et domaines des maths et de l'info.
 - ▶ Construction algorithmique des preuves.
 - ▶ Construction algorithmique des modèles d'une théorie.
 - ▶ Résultats d'impossibilité.
- ▶ Applications :
 - ▶ Preuve assistée par ordinateur.
 - ▶ Découverte des bugs dans des logiciels.